

C/C++

```
/* Miguel Amiel Chen Xu Grupo 81 Equipo 107 100525454@alumnos.uc3m.es 100522395@alumnos.uc3m.es */

%{
    // SECTION 1 Declarations for C-Bison #include <stdio.h> #include <ctype.h>
    tolower() #include <string.h> // strcmp() #include <stdlib.h> // exit()

#define FF fflush(stdout); // to force immediate printing

int yylex () ; void yyerror (char *) ; char *my_malloc (int) ;

// Not needed using Direct Translation: char *gen_code (char *) ; char *int_to_string (int) ; char *char_to_string (char) ;

char temp [2048] ;

// Definitions for explicit attributes

typedef struct s_attr { int value ; // - Numeric value of a NUMBER char *code ; // - to pass IDENTIFIER names, and other
translations } t_attr ;

#define YYSTYPE t_attr // stack of PDA has type t_attr

%}

// Definitions for explicit attributes

%token NUMBER
%token IDENTIF // Identifier=variable %token STRING // token for string type %token MAIN // token for
keyword main // main is not predefined in Lisp but we will use it as a keyword0+
```

```

%token WHILE          // token for keyword while %token LOOP %token DO %token SETQ %token SETF %token DEFUN
%token PRINT
%token PRINC
%token AND
%token OR
%token NOT
%token NOT_EQUAL
%token LESS_EQUAL
%token GREATER_EQUAL
%token MOD
%token IF
%token PROGN

// %prec section not needed in LISP

%%
// Section 3 Grammar - Semantic Actions axiom:
// A Lisp program contains a sequence of at least one expression ;

exprSeq:      expression1          { ; }      // level 1 expressions must exclude specific level 2 expressions.
ToDo in the Future r_exprSeq      { ; } ;

r_exprSeq:    exprSeq              { ; } | /* lambda */
                                                    { ; } ;

expression1:  expression           { ; } // Lisp can evaluate arithmetical (and similar) expressions in REPL
mode // REPL Mode should print out the evaluated expressions ==> Future TODO for the Forth translation

          | '(' SETQ IDENTIF number ') ' { printf(" variable %s %s !\n", $3.code, $3.code);} // This is the declaration
of a variable which in Forth has to be of global scope

```

```
| '(' SETF IDENTIF expression ')' { printf(" %s !\n", $3.code);} // Using a variable as receiver requires  
adding the store operator (!) in Forth
```

```
| '(' PRINT STRING ')' { printf(" .\n %s\n cr", $3.code);}
```

```
| '(' PRINC princ_param ')' { ; } // Princ should be able to print both expressions and strings
```

```
| '(' PROGN exprSeq ')' { ; }
```

```
| '(' MAIN ')' { printf (" main\n") ; } // call to the main function
```

```
| '(' DEFUN MAIN  
  '(' ')' exprSeq ')' { printf(": main ");  
                      { printf(" ;\n"); }
```

```
| '(' DEFUN IDENTIF  
  '(' ')' exprSeq ')' { printf(": %s ", $3.code);}  
                      { printf (" ;\n") ; }
```

// In real Lisp some expressions like if or Loop-While-Do are only permitted inside defun definitions (level 2 expressions)
==> Future ToDo // Level 1 and common expressions (arithmetic etc.) are also permitted inside a defun definition

```
| '(' LOOP WHILE  
  expression  
  DO exprSeq ')' { printf(" begin ");  
                  { printf(" while "); }  
                  { printf(" repeat "); }
```

```
| '(' ifHead expression1 ')' { printf (" THEN\n") ; } // If Expression then Expression1  
// ifHead is used to avoid conflicts through partial
```

factorization

```
| '(' ifHead expression1  
  expression2 ')' { printf (" ELSE\n") ; } // If Expression then Expression1 else Expression1
```

```

        expression1 ')'          { printf (" THEN\n") ; }    // more than one expression per then or else branch
are only allowed nesting them within a PROGN expression
    ;

princ_param:

    expression { printf(" . ");}

    | STRING { printf(".\n" %s", $1.code);} ;

ifHead:      IF expression          { printf (" IF ") ; }    // Real Lisp restricts if conditions to Boolean
type expressions (excluding base operands?) ==> Future TODO ;

expression:  operand                { ; }                    // Common expressions combine arithmetic,
relational and boolean expressions, including base operands.

    | '(' '-' expression expression ')' { printf (" - ") ; }    // binary minus operator

/* - * / MOD AND OR > < GE LE ... NOT */ | '(' AND  expression expression ')' { printf(" %s ", $2.code);}

    | '(' OR   expression expression ')' { printf(" %s ", $2.code);}

    | '(' NOT  expression ')'           { printf(" 0= ");}

    | '(' NOT_EQUAL expression expression ')' { printf(" = 0= ");}

    | '(' '='  expression expression ')'   { printf(" = ");}

    | '(' '<'  expression expression ')' { printf(" < ");}

```

```

| '(' LESS_EQUAL expression expression ')' { printf(" %s ", $2.code);}

| '(' '>' expression expression ')' { printf(" > ");}

| '(' GREATER_EQUAL expression expression ')' { printf(" %s ", $2.code);}

| '(' MOD expression expression ')' { printf(" %s ", $2.code);}
| '(' '+' expression expression ')' { printf(" + ");}
| '(' '*' expression expression ')' { printf(" * ");}
| '(' '/' expression expression ')' { printf(" / ");}

| '(' '-' expression ')' { printf(" negate "); } // Unary minus operator in Lisp
;

operand: IDENTIF { printf(" %s @ ", $1.code) ; } // To use a variable as an operand requires
adding the fetch operator (@) | number { ; } ;

number: NUMBER { printf(" %d ", $1.value) ; } // number is an auxiliary Non Terminal to
be used in the setq initialization ;

%%
// SECTION 4 Code in C

int n_line = 1 ;

void yyerror (char *message) { fprintf (stderr, "%s in line %d\n", message, n_line) ; printf ( "\n" ) ; }

char *int_to_string (int n) { char temp [1024] ;

sprintf (temp, "%d", n) ;

```

```

return gen_code (temp) ;

}

char *char_to_string (char c) { char temp [1024] ;

sprintf (temp, "%c", c) ;

return gen_code (temp) ;

}

char *gen_code (char *name) // copy the argument to an
{ // string in dynamic memory
char *p ; int l ;

l = strlen (name)+1 ;
p = (char *) my_malloc (l) ;
strcpy (p, name) ;

return p ;

}

char *my_malloc (int nbytes) // reserve n bytes of dynamic memory { char *p ; static long int nb = 0 ; // used to
count the memory
static int nv = 0 ; // required in total

p = malloc (nbytes) ;
if (p == NULL) {

```

```

    fprintf (stderr, "No memory left for additional %d bytes\n", nbytes) ;
    fprintf (stderr, "%ld bytes reserved in %d calls \n", nb, nv) ;
    exit (0) ;
}
nb += (long) nbytes ;
nv++ ;

return p ;

}

// / Keyword Section / //

typedef struct s_keyword { // for the reserved words of C
char *name ; int token ; } t_keyword ;

t_keyword keywords [] = {
// define the keywords "main",
MAIN,      // and their associated token
"defun",    DEFUN,
"print",    PRINT,
"setq",     SETQ,
"setf",     SETF,
"princ",    PRINC,
"loop",     LOOP,
"while",    WHILE,
"do",       DO,
"and",      AND,
"or",       OR,
"not",      NOT,

```

```

"/=",          NOT_EQUAL,
"<=",          LESS_EQUAL,
">=",          GREATER_EQUAL,
"mod",         MOD,
"if",          IF,
"progn",       PROGN,
NULL, 0        // 0 to mark the end of the table } ;

t_keyword *search_keyword (char *symbol_name) {                // Search symbol names in the keyword table // and
return a pointer to token register int i ; t_keyword *sim ;

i = 0 ;
sim = keywords ;
while (sim [i].name != NULL) {
    if (strcmp (sim [i].name, symbol_name) == 0) {
        // strcmp(a, b) returns == 0 if a==b
        return &(sim [i]) ;
    }
    i++ ;
}

return NULL ;

}

// / Section for the Lexical Analyzer / //

int yylex () { int i ; unsigned char c ; unsigned char cc ; char expandable_ops [] = " !<>=|%&/- * + " ; char temp_str [256] ;
t_keyword *symbol ;

```

```

do {
    c = getchar () ;
    if (c == '#') { // Ignore the lines starting with # (#define, #include)
        do { // WARNING that it may malfunction if a line contains #
            c = getchar () ;
        } while (c != '\n') ;
    }
    if (c == '/') { // character / can be the beginning of a comment.
        cc = getchar () ;
        if (cc != '/') { // If the following char is / is a comment, but....
            ungetc (cc, stdin) ;
        } else {
            c = getchar () ; // ...
            if (c == '@') { // Lines starting with //@ are transcribed
                do { // This is inline code (embedded code in C).
                    c = getchar () ;
                    putchar (c) ;
                } while (c != '\n' && c != EOF) ;
                if (c == EOF) {
                    ungetc (c, stdin) ;
                }
            } else { // ==> comment, ignore the line
                while (c != '\n') {
                    c = getchar () ;
                }
            }
        }
    }
}
if (c == '\n')
    n_line++ ;

```

```

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

if (c == '\\') {
    i = 0 ;
    do {
        c = getchar () ;
        temp_str [i++] = c ;
    } while (c != '\\' && i < 255) ;
    if (i == 256) {
        printf ("WARNING: string with more than 255 characters in line %d\n", n_line) ;
    } // we should read until the next ", but, what if it is missing?
    temp_str [--i] = '\\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;
    scanf ("%d", &yylval.value) ;

    //      printf ("\nDEV: NUMBER %d\n", yylval.value) ;
    return NUMBER ; }

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0 ;
    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        temp_str [i++] = tolower (c) ; // ALL TO SMALL LETTERS
        c = getchar () ;
    }
}

```

```

temp_str [i] = '\0' ; // End of string
ungetc (c, stdin) ; // return excess char

yyval.code = gen_code (temp_str) ;
symbol = search_keyword (yyval.code) ;
if (symbol == NULL) { // is not reserved word -> iderntifrier

//          printf ("\nDEV: IDENTIF %s\n", yyval.code) ;    // PARA DEPURAR return (IDENTIF) ; } else { //
printf ("\nDEV: OTRO %s\n", yyval.code) ;          // PARA DEPURAR return (symbol->token) ; } }

if (strchr (expandable_ops, c) != NULL) { // // look for c in expandable_ops
    cc = getchar () ;
    sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
    symbol = search_keyword (temp_str) ;
    if (symbol == NULL) {
        ungetc (cc, stdin) ;
        yyval.code = NULL ;
        return (c) ;
    } else {
        yyval.code = gen_code (temp_str) ; // although it is not used
        return (symbol->token) ;
    }
}

//    printf ("\nDEV: LITERAL %d %#c#\n", (int) c, c) ;    // PARA DEPURAR if (c == EOF || c == 255 || c == 26) { //
printf ("tEOF ") ;          // PARA DEPURAR return (0) ; }

return c ;

}

```

```
int main () { yyparse () ; }
```